

COMP102 Lecture 13: SQL Foundations

SELECT, WHERE, JOIN, GROUP BY, ORDER BY, indexing basics

Jalal Maaouni

June 29, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

Plan

- 1 From database design to SQL
- 2 From SQL databases to SQLite
- 3 Basic querying
- 4 Ordering
- 5 Joins
- 6 Aggregation
- 7 Indexing basics
- 8 Recap

FROM DATABASE DESIGN TO SQL

Where we stopped last time

- ▶ We learned why databases are needed: persistence, sharing, scale, and integrity.
- ▶ We represented data using relations: tables with rows and columns.
- ▶ We used keys to identify rows and connect tables.
- ▶ We mapped an ER model into a relational schema.
- ▶ We normalized tables so each fact lives in one place.

Last lecture: how should data be structured?

This lecture: how do we ask questions from that structure?

The schema we will keep using

Table	Columns	Role
Student	id, name, major	student facts
Course	code, title, credits, department	course facts
Enrollment	student_id, course_code, grade	who takes what

This is the university schema from the previous lecture.

Why the data was split

- ▶ Student names belong in Student.
- ▶ Course titles belong in Course.
- ▶ Grades belong in Enrollment.
- ▶ The many-to-many relationship needs a separate table.

Normalization avoids repeated facts. SQL lets us recombine those facts when we need answers.

Design questions vs query questions

Design question	Query question
What tables do we need?	Which students are in CS?
What is the primary key?	Which courses does Amal take?
Where does the foreign key go?	How many students are in each course?
How do we avoid redundancy?	Which course has the highest average grade?

SQL is the language for query questions.

From relation schema to SQL table

Last lecture notation:

```
1 Student(id, name, major)
```

SQL version:

```
1 CREATE TABLE Student (  
2     id INTEGER PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     major TEXT  
5 );
```

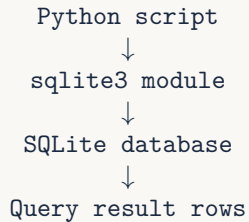
The abstract schema becomes executable instructions for the DBMS.

The role of SQL

- ▶ SQL creates tables.
- ▶ SQL inserts data.
- ▶ SQL retrieves selected rows and columns.
- ▶ SQL joins related tables.
- ▶ SQL summarizes, sorts, and filters results.
- ▶ SQL can ask the DBMS to use indexes for faster lookup.

SQL is how we talk to a relational database.

Our workflow today



Python will run the examples. SQLite will store the tables and execute the SQL.

Today's guiding questions

1. How do we retrieve columns from a table?
2. How do we filter rows?
3. How do we sort results?
4. How do we recombine normalized tables?
5. How do we summarize many rows?
6. How do indexes make some queries faster?

Every new SQL keyword today answers one of these questions.

FROM SQL DATABASES TO SQLITE

SQL is a language, not one specific product

Many database systems understand SQL, but they differ in installation, scale, administration, features, and typical use cases.

- ▶ PostgreSQL
- ▶ MySQL / MariaDB
- ▶ SQLite
- ▶ Oracle Database
- ▶ Microsoft SQL Server
- ▶ DuckDB

For this course, the goal is not database administration. The goal is to learn how to query relational data.

Choosing a database for COMP102

System	Typical use
PostgreSQL	serious server-side applications, strong SQL support
MySQL / MariaDB	web applications, production systems
SQLite	local files, small apps, scripts, learning, prototypes
DuckDB	local analytics and data science workloads
Oracle / SQL Server	large enterprise systems

We will use SQLite because it runs directly from Python and requires no server setup.

Why SQLite?

- ▶ SQLite is included with Python through the `sqlite3` module.
- ▶ It can store a database in memory or in a single local file.
- ▶ It supports the SQL foundations we need: `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY`, and indexes.
- ▶ It is simple enough for lectures, labs, scripts, and small applications.

SQLite is the engine. SQL is the language. Python is how we will run the examples.

Open an in-memory SQLite database

```
1 import sqlite3
2
3 con = sqlite3.connect(":memory:")
4 con.execute("PRAGMA foreign_keys = ON")
```

- ▶ `:memory:` creates a temporary database in RAM.
- ▶ The database disappears when the program ends.
- ▶ This is useful for clean lecture examples and tests.

Open a file-based SQLite database

```
1 import sqlite3
2
3 con = sqlite3.connect("university.db")
4 con.execute("PRAGMA foreign_keys = ON")
```

- ▶ `university.db` is saved on disk.
- ▶ The data survives after the program stops.
- ▶ This is closer to how a real application stores local data.

A helper function for the lecture

```
1 def show(sql, params=()):
2     print("\nSQL:")
3     print(sql.strip())
4
5     print("\nResult:")
6     for row in con.execute(sql, params):
7         print(row)
```

Python sends a SQL string to SQLite. SQLite executes the query and returns rows.

Testing the connection

```
1 show("""  
2 SELECT 1 + 1;  
3 """)
```

Expected output:

```
1 SQL:  
2 SELECT 1 + 1;  
3  
4 Result:  
5 (2,)
```

Create the main tables

```
1  con.executescript("""
2  DROP TABLE IF EXISTS Enrollment;
3  DROP TABLE IF EXISTS Course;
4  DROP TABLE IF EXISTS Student;
5
6  CREATE TABLE Student (
7      id INTEGER PRIMARY KEY,
8      name TEXT NOT NULL,
9      major TEXT
10 );
11
12 CREATE TABLE Course (
13     code TEXT PRIMARY KEY,
14     title TEXT NOT NULL,
15     credits INTEGER NOT NULL,
16     department TEXT NOT NULL
17 );
18 """)
```

Create the relationship table

```
1 con.executescript("""
2 CREATE TABLE Enrollment (
3     student_id INTEGER REFERENCES Student(id),
4     course_code TEXT REFERENCES Course(code),
5     grade REAL,
6     PRIMARY KEY (student_id, course_code)
7 );
8 """)
```

Enrollment connects students and courses.

SQL statement: DROP TABLE IF EXISTS

```
1 DROP TABLE IF EXISTS Enrollment;
```

- ▶ DROP TABLE deletes a table.
- ▶ IF EXISTS avoids an error if the table is already absent.
- ▶ Useful when rerunning the same script many times.

SQL statement: CREATE TABLE

```
1 CREATE TABLE Student (  
2     id INTEGER PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     major TEXT  
5 );
```

- ▶ CREATE TABLE defines a new relation.
- ▶ The table name is Student.
- ▶ Parentheses contain the column definitions.

Column definitions

```
1 id INTEGER PRIMARY KEY,  
2 name TEXT NOT NULL,  
3 major TEXT
```

- ▶ Each line defines one column.
- ▶ Format: name type constraints.
- ▶ Commas separate column definitions.

SQLite data types

```
1 id INTEGER
2 name TEXT
3 grade REAL
```

- ▶ INTEGER: whole numbers.
- ▶ TEXT: strings.
- ▶ REAL: decimal numbers.

Constraint: PRIMARY KEY

```
1 id INTEGER PRIMARY KEY
```

- ▶ PRIMARY KEY identifies each row.
- ▶ Values must be unique.
- ▶ Values cannot be NULL.

Constraint: NOT NULL

```
1 name TEXT NOT NULL
```

- ▶ NOT NULL forbids missing values.
- ▶ Every student must have a name.
- ▶ But major TEXT may be NULL.

Foreign keys: REFERENCES

```
1 student_id INTEGER REFERENCES Student(id),  
2 course_code TEXT REFERENCES Course(code)
```

- ▶ REFERENCES creates a foreign key.
- ▶ student_id must match an existing Student.id.
- ▶ course_code must match an existing Course.code.

Composite primary key

```
1 PRIMARY KEY (student_id, course_code)
```

- ▶ The key is the pair (student_id, course_code).
- ▶ A student cannot enroll in the same course twice.
- ▶ The same student may enroll in many courses.
- ▶ The same course may contain many students.

End of statement: semicolon

```
1 CREATE TABLE Course (  
2     code TEXT PRIMARY KEY,  
3     title TEXT NOT NULL  
4 );
```

- ▶ The semicolon ; ends a SQL statement.
- ▶ A script may contain many SQL statements.
- ▶ `executescript` runs the whole script in order.

The schema as tables

Table	Main columns	Role
Student	id, name, major	stores students
Course	code, title, credits, department	stores courses
Enrollment	student_id, course_code, grade	connects students to courses

This is the schema from the previous lecture, now written as executable SQL.

Prepare sample data

```
1 students = [  
2     (1, "Amal", "CS"),  
3     (2, "Yassine", "Math"),  
4     (3, "Salma", "CS"),  
5     (4, "Nora", None),  
6 ]  
7 courses = [  
8     ("CS200", "Algorithms", 4, "CS"),  
9     ("MA101", "Calculus", 3, "Math"),  
10    ("CS150", "Programming", 4, "CS"),  
11    ("PH100", "Physics", 3, "Physics"),  
12 ]  
13 enrollments = [  
14     (1, "CS200", 17.0),  
15     (1, "MA101", 14.0),  
16     (2, "MA101", 16.0),  
17     (3, "CS200", 18.0),  
18     (3, "CS150", 15.5),  
19 ]
```


Insert students and courses

```
1 con.executemany("""
2 INSERT INTO Student(id, name, major)
3 VALUES (?, ?, ?);
4 """, students)
5
6 con.executemany("""
7 INSERT INTO Course(code, title, credits, department)
8 VALUES (?, ?, ?, ?);
9 """, courses)
```

The question marks are placeholders. Python sends the values separately from the SQL text.

Insert enrollments

```
1 con.executemany("""  
2 INSERT INTO Enrollment(student_id, course_code, grade)  
3 VALUES (?, ?, ?);  
4 """, enrollments)  
5  
6 con.commit()
```

The enrollment table stores the many-to-many relationship between students and courses.

SQL statement: INSERT INTO

```
1 INSERT INTO Student(id, name, major)
2 VALUES (?, ?, ?);
```

- ▶ INSERT INTO adds new rows to a table.
- ▶ Student is the target table.
- ▶ (id, name, major) specifies which columns receive values.

SQL statement: VALUES

```
1 VALUES (?, ?, ?);
```

- ▶ VALUES gives the data for one new row.
- ▶ The order must match the column list.
- ▶ Here: first value goes to `id`, second to `name`, third to `major`.

Python placeholders: ?

```
1 con.executemany("""  
2 INSERT INTO Student(id, name, major)  
3 VALUES (?, ?, ?);  
4 """, students)
```

- ▶ Each ? is a placeholder.
- ▶ Python replaces placeholders with values from each tuple.
- ▶ Do not build SQL values with string formatting.

Python command: executemany

```
1 con.executemany(sql_query, rows)
```

- ▶ executemany runs the same SQL command many times.
- ▶ Each tuple in rows supplies one set of values.
- ▶ Useful for inserting a list of records.

Example: inserting several students

```
1 students = [  
2     (1, "Amal", "CS"),  
3     (2, "Yassine", "Math"),  
4     (3, "Salma", "CS"),  
5 ]  
6  
7 con.executemany("""  
8 INSERT INTO Student(id, name, major)  
9 VALUES (?, ?, ?);  
10 """, students)
```

Python command: `commit`

```
1 con.commit()
```

- ▶ `commit` saves the transaction.
- ▶ Without committing, changes may not be permanently stored.
- ▶ For an in-memory database, it still marks the current changes as accepted.

Check the student table

```
1 show("""  
2 SELECT *  
3 FROM Student;  
4 """)
```

Expected output:

```
1 (1, 'Amal', 'CS')  
2 (2, 'Yassine', 'Math')  
3 (3, 'Salma', 'CS')  
4 (4, 'Nora', None)
```

Check the course table

```
1 show("""  
2 SELECT *  
3 FROM Course;  
4 """)
```

Expected output:

```
1 ('CS200', 'Algorithms', 4, 'CS')  
2 ('MA101', 'Calculus', 3, 'Math')  
3 ('CS150', 'Programming', 4, 'CS')  
4 ('PH100', 'Physics', 3, 'Physics')
```

Check the enrollment table

```
1 show("""  
2 SELECT *  
3 FROM Enrollment;  
4 """)
```

Expected output:

```
1 (1, 'CS200', 17.0)  
2 (1, 'MA101', 14.0)  
3 (2, 'MA101', 16.0)  
4 (3, 'CS200', 18.0)  
5 (3, 'CS150', 15.5)
```

Foreign keys are enforced

```
1 con.execute("""
2 INSERT INTO Enrollment(student_id, course_code, grade)
3 VALUES (99, 'CS200', 12.0);
4 """)
```

Expected error:

```
1 sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

The database refuses an enrollment for a student that does not exist.

Primary keys are enforced

```
1 con.execute("""  
2 INSERT INTO Student(id, name, major)  
3 VALUES (1, 'Imane', 'Physics');  
4 """)
```

Expected error:

```
1 sqlite3.IntegrityError: UNIQUE constraint failed: Student.id
```

The database refuses two students with the same primary key.

What we now have

```
1 # We now have:  
2 # - a SQLite connection  
3 # - three relational tables  
4 # - primary keys  
5 # - foreign keys  
6 # - sample data  
7 # - a helper function for showing query results
```

Next step: learn the SQL clauses that let us ask questions from these tables.

BASIC QUERYING

First query: show all students

```
1 show("""  
2 SELECT *  
3 FROM Student;  
4 """)
```

A query asks the database a question and returns rows as the answer.

SQL keyword: SELECT

```
1 SELECT name, major  
2 FROM Student;
```

- ▶ SELECT chooses which columns appear in the result.
- ▶ name, major means: return only these two columns.
- ▶ SELECT * means: return all columns.

Selecting specific columns

```
1 show("""  
2 SELECT name, major  
3 FROM Student;  
4 """)
```

Expected output:

```
1 ('Amal', 'CS')  
2 ('Yassine', 'Math')  
3 ('Salma', 'CS')  
4 ('Nora', None)
```

SQL keyword: FROM

```
1 SELECT name, major  
2 FROM Student;
```

- ▶ FROM chooses the table we query.
- ▶ Here, rows come from the Student table.
- ▶ A basic query needs at least SELECT and FROM.

Query another table

```
1 show("""
2 SELECT code, title, credits
3 FROM Course;
4 """)
```

Expected output:

```
1 ('CS200', 'Algorithms', 4)
2 ('MA101', 'Calculus', 3)
3 ('CS150', 'Programming', 4)
4 ('PH100', 'Physics', 3)
```

SQL keyword: WHERE

```
1 SELECT name  
2 FROM Student  
3 WHERE major = 'CS';
```

- ▶ WHERE filters rows.
- ▶ Only rows satisfying the condition are returned.
- ▶ Here, we keep only students whose major is 'CS'.

Filtering students

```
1 show("""  
2 SELECT name  
3 FROM Student  
4 WHERE major = 'CS';  
5 """)
```

Expected output:

```
1 ('Amal',)  
2 ('Salma',)
```

Filtering courses

```
1 show("""
2 SELECT title, credits
3 FROM Course
4 WHERE credits >= 4;
5 """)
```

Expected output:

```
1 ('Algorithms', 4)
2 ('Programming', 4)
```

Common comparison operators

Operator	Meaning
=	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Combining conditions: AND

```
1 show("""  
2 SELECT title  
3 FROM Course  
4 WHERE department = 'CS' AND credits = 4;  
5 """)
```

Expected output:

```
1 ('Algorithms',)  
2 ('Programming',)
```

Combining conditions: OR

```
1 show("""
2 SELECT name, major
3 FROM Student
4 WHERE major = 'CS' OR major = 'Math';
5 """)
```

Expected output:

```
1 ('Amal', 'CS')
2 ('Yassine', 'Math')
3 ('Salma', 'CS')
```

SQL value: NULL

```
1 major TEXT
```

- ▶ NULL means missing, unknown, or not applicable.
- ▶ It is not zero.
- ▶ It is not the empty string.
- ▶ In our data, Nora has no declared major yet.

Incorrect NULL comparison

```
1 show("""  
2 SELECT name  
3 FROM Student  
4 WHERE major = NULL;  
5 """)
```

Expected output:

```
1 # No rows
```

NULL is not compared with =.

Correct keyword: IS NULL

```
1 show("""  
2 SELECT name  
3 FROM Student  
4 WHERE major IS NULL;  
5 """)
```

Expected output:

```
1 ('Nora',)
```

Correct keyword: IS NOT NULL

```
1 show("""  
2 SELECT name, major  
3 FROM Student  
4 WHERE major IS NOT NULL;  
5 """)
```

Expected output:

```
1 ('Amal', 'CS')  
2 ('Yassine', 'Math')  
3 ('Salma', 'CS')
```

Parameterized query

```
1 major = "CS"
2
3 show("""
4 SELECT name
5 FROM Student
6 WHERE major = ?;
7 """, (major,))
```

- ▶ The ? is a placeholder.
- ▶ Python sends "CS" separately.
- ▶ SQLite safely combines the SQL command and the value.

Why not use string formatting?

```
1 major = "CS"
2
3 sql = f"""
4 SELECT name
5 FROM Student
6 WHERE major = '{major}';
7 """
```

This works for harmless examples, but it is a bad habit. Values should be passed using placeholders.

Parameterized course query

```
1 min_credits = 4
2
3 show("""
4 SELECT code, title
5 FROM Course
6 WHERE credits >= ?;
7 """, (min_credits,))
```

Expected output:

```
1 ('CS200', 'Algorithms')
2 ('CS150', 'Programming')
```

Basic query pattern

```
1 SELECT column1, column2
2 FROM table_name
3 WHERE condition;
```

- ▶ SELECT: what columns should appear?
- ▶ FROM: which table do we read?
- ▶ WHERE: which rows do we keep?

Your turn

Write a SQL query for each question.

1. Show all course titles.
2. Show the names of students whose major is Math.
3. Show the courses with exactly 3 credits.
4. Show students whose major is missing.

Use only SELECT, FROM, WHERE, and IS NULL.

ORDERING

Tables are not ordered

```
1 show("""  
2 SELECT name, major  
3 FROM Student;  
4 """)
```

A table is a set of rows. Sets do not have an inherent order.

SQL keyword: ORDER BY

```
1 SELECT name, major  
2 FROM Student  
3 ORDER BY name;
```

- ▶ ORDER BY sorts the query result.
- ▶ It does not change the table itself.
- ▶ Sorting is requested explicitly at query time.

Sort students by name

```
1 show("""  
2 SELECT name, major  
3 FROM Student  
4 ORDER BY name;  
5 """)
```

Expected output:

```
1 ('Amal', 'CS')  
2 ('Nora', None)  
3 ('Salma', 'CS')  
4 ('Yassine', 'Math')
```

Ascending and descending order

```
1 ORDER BY credits ASC;  
2 ORDER BY credits DESC;
```

- ▶ ASC: ascending order, from smallest to largest.
- ▶ DESC: descending order, from largest to smallest.
- ▶ If omitted, SQL uses ascending order by default.

Sort courses by credits

```
1 show("""  
2 SELECT title, credits  
3 FROM Course  
4 ORDER BY credits DESC;  
5 """)
```

Expected output:

```
1 ('Algorithms', 4)  
2 ('Programming', 4)  
3 ('Calculus', 3)  
4 ('Physics', 3)
```

Sorting ties

```
1 show("""  
2 SELECT title, credits  
3 FROM Course  
4 ORDER BY credits DESC, title ASC;  
5 """)
```

- ▶ First sort by credits in descending order.
- ▶ If two courses have the same credits, sort them by title.

SQL keyword: LIMIT

```
1 SELECT title, credits
2 FROM Course
3 ORDER BY credits DESC
4 LIMIT 2;
```

- ▶ LIMIT keeps only a fixed number of rows.
- ▶ It is usually used after ORDER BY.
- ▶ Without ORDER BY, LIMIT may return arbitrary rows.

Top courses by credits

```
1 show("""  
2 SELECT title, credits  
3 FROM Course  
4 ORDER BY credits DESC  
5 LIMIT 2;  
6 """)
```

Expected output:

```
1 ('Algorithms', 4)  
2 ('Programming', 4)
```

Best grades

```
1 show("""
2 SELECT student_id, course_code, grade
3 FROM Enrollment
4 ORDER BY grade DESC
5 LIMIT 3;
6 """)
```

Expected output:

```
1 (3, 'CS200', 18.0)
2 (1, 'CS200', 17.0)
3 (2, 'MA101', 16.0)
```

Ordering with filtering

```
1 show("""
2 SELECT title, credits
3 FROM Course
4 WHERE department = 'CS'
5 ORDER BY title;
6 """)
```

Expected output:

```
1 ('Algorithms', 4)
2 ('Programming', 4)
```

Query pattern with ordering

```
1 SELECT column1, column2
2 FROM table_name
3 WHERE condition
4 ORDER BY column1 ASC
5 LIMIT n;
```

- ▶ WHERE chooses the rows.
- ▶ ORDER BY sorts the remaining rows.
- ▶ LIMIT keeps only the first n rows.

Your turn

Write a SQL query for each question.

1. Show all students sorted by name.
2. Show all courses sorted from highest credits to lowest credits.
3. Show the two courses with the highest number of credits.
4. Show the three best grades in Enrollment.

Use SELECT, FROM, WHERE, ORDER BY, and LIMIT.

JOINS

Why normalized data needs joins

- ▶ Student stores student facts.
- ▶ Course stores course facts.
- ▶ Enrollment stores who takes what.

No single table contains student names, course titles, and grades together. To answer such questions, we need joins.

The question we want to answer

Question:

Which students are enrolled in which courses, and what grades did they get?

```
1 -- We want:  
2 -- student name / course title / grade
```

The answer requires columns from three different tables.

The enrollment table alone is not enough

```
1 show("""
2 SELECT student_id, course_code, grade
3 FROM Enrollment
4 ORDER BY student_id, course_code;
5 """)
```

Expected output:

```
1 (1, 'CS200', 17.0)
2 (1, 'MA101', 14.0)
3 (2, 'MA101', 16.0)
4 (3, 'CS200', 18.0)
5 (3, 'CS150', 15.5)
```

What is missing?

Column	Problem
student_id	identifies the student, but does not show the name
course_code	identifies the course, but does not show the title
grade	already available in Enrollment

A foreign key is useful because it tells us which row to look up in another table.

SQL keyword: JOIN

```
1 SELECT Student.name, Enrollment.grade
2 FROM Enrollment
3 JOIN Student ON Enrollment.student_id = Student.id;
```

- ▶ JOIN combines rows from two tables.
- ▶ It creates result rows by matching related rows.
- ▶ The matching rule is written after ON.

SQL keyword: ON

```
1 ON Enrollment.student_id = Student.id
```

- ▶ ON gives the condition for matching rows.
- ▶ Here, an enrollment row matches the student whose id equals student_id.
- ▶ This follows the foreign-key relationship.

Join intuition

Enrollment row	Matching Student row	Result
(1, CS200, 17.0)	(1, Amal, CS)	(Amal, CS200, 17.0)
(2, MA101, 16.0)	(2, Yassine, Math)	(Yassine, MA101, 16.0)

A join does not guess. It matches rows using the condition after ON.

Join students with enrollments

```
1 show("""
2 SELECT Student.name, Enrollment.course_code, Enrollment.grade
3 FROM Enrollment
4 JOIN Student ON Enrollment.student_id = Student.id
5 ORDER BY Student.name, Enrollment.course_code;
6 """)
```

Expected output:

```
1 ('Amal', 'CS200', 17.0)
2 ('Amal', 'MA101', 14.0)
3 ('Salma', 'CS150', 15.5)
4 ('Salma', 'CS200', 18.0)
5 ('Yassine', 'MA101', 16.0)
```

Qualified column names

```
1 Student.name  
2 Enrollment.grade  
3 Course.title
```

- ▶ A qualified column name has the form `table.column`.
- ▶ It makes the source table explicit.
- ▶ It avoids ambiguity when several tables are used.

In join queries, prefer qualified column names.

SQL keyword: INNER JOIN

```
1 FROM Enrollment  
2 INNER JOIN Student ON Enrollment.student_id = Student.id
```

- ▶ INNER JOIN keeps only matching rows.
- ▶ In SQLite, JOIN means INNER JOIN by default.
- ▶ Rows without a match are not returned.

JOIN and INNER JOIN are equivalent here

```
1 FROM Enrollment
2 JOIN Student ON Enrollment.student_id = Student.id
```

is the same as:

```
1 FROM Enrollment
2 INNER JOIN Student ON Enrollment.student_id = Student.id
```

We will usually write JOIN, but the idea is an inner join.

Why course PH100 does not appear

```
1 show("""
2 SELECT Course.code, Course.title, Enrollment.student_id
3 FROM Course
4 JOIN Enrollment ON Course.code = Enrollment.course_code
5 ORDER BY Course.code;
6 """)
```

PH100 has no enrollment row. An inner join keeps only courses that have a matching enrollment.

SQL keyword: AS

```
1 FROM Enrollment AS e
2 JOIN Student AS s ON e.student_id = s.id
```

- ▶ AS gives a temporary alias to a table.
- ▶ Enrollment AS e means: refer to Enrollment as e.
- ▶ Aliases make join queries shorter and easier to read.

Same join using aliases

```
1 show("""
2 SELECT s.name, e.course_code, e.grade
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 ORDER BY s.name, e.course_code;
6 """)
```

Expected output:

```
1 ('Amal', 'CS200', 17.0)
2 ('Amal', 'MA101', 14.0)
3 ('Salma', 'CS150', 15.5)
4 ('Salma', 'CS200', 18.0)
5 ('Yassine', 'MA101', 16.0)
```

Alias convention

Table	Alias
Student	s
Course	c
Enrollment	e

Aliases exist only inside the query. They do not rename the actual tables.

Join students and courses through enrollment

```
1 FROM Enrollment AS e
2 JOIN Student AS s ON e.student_id = s.id
3 JOIN Course AS c ON e.course_code = c.code
```

- ▶ Enrollment gives the pairs.
- ▶ Student gives the student names.
- ▶ Course gives the course titles.

Join across three tables

```
1 show("""
2 SELECT s.name, c.title, e.grade
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 JOIN Course AS c ON e.course_code = c.code
6 ORDER BY s.name, c.title;
7 """)
```

Expected output:

```
1 ('Amal', 'Algorithms', 17.0)
2 ('Amal', 'Calculus', 14.0)
3 ('Salma', 'Algorithms', 18.0)
4 ('Salma', 'Programming', 15.5)
5 ('Yassine', 'Calculus', 16.0)
```

Reading a three-table join

Clause	Meaning
FROM Enrollment AS e	start from enrollment rows
JOIN Student AS s	attach the matching student row
ON e.student_id = s.id	match enrollment to student
JOIN Course AS c	attach the matching course row
ON e.course_code = c.code	match enrollment to course

Filter after joining

Question: which students are taking Algorithms?

```
1 show("""
2 SELECT s.name
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 JOIN Course AS c ON e.course_code = c.code
6 WHERE c.title = 'Algorithms'
7 ORDER BY s.name;
8 """)
```

Expected output:

```
1 ('Amal',)
2 ('Salma',)
```

Filter by student after joining

Question: which courses does Amal take?

```
1 show("""
2 SELECT c.title, e.grade
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 JOIN Course AS c ON e.course_code = c.code
6 WHERE s.name = 'Amal'
7 ORDER BY c.title;
8 """)
```

Expected output:

```
1 ('Algorithms', 17.0)
2 ('Calculus', 14.0)
```

Filter by department after joining

Question: show all enrollments in CS courses.

```
1 show("""
2 SELECT s.name, c.title, e.grade
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 JOIN Course AS c ON e.course_code = c.code
6 WHERE c.department = 'CS'
7 ORDER BY c.title, s.name;
8 """)
```

Expected output:

```
1 ('Amal', 'Algorithms', 17.0)
2 ('Salma', 'Algorithms', 18.0)
3 ('Salma', 'Programming', 15.5)
```

Join pattern

```
1 SELECT selected_columns
2 FROM first_table AS a
3 JOIN second_table AS b ON a.foreign_key = b.primary_key
4 WHERE condition
5 ORDER BY column;
```

- ▶ FROM chooses the starting table.
- ▶ JOIN adds another table.
- ▶ ON explains how rows match.

Common mistake: missing join condition

```
1 SELECT Student.name, Course.title  
2 FROM Student, Course;
```

This does not match students to their courses. It combines every student with every course.

Cartesian product

- ▶ A Cartesian product pairs every row from one table with every row from another table.
- ▶ If Student has 4 rows and Course has 4 rows, the result has 16 rows.
- ▶ Most of these pairs are meaningless.

A join without a correct matching condition is usually a bug.

Cartesian product in Python

```
1 show("""  
2 SELECT Student.name, Course.title  
3 FROM Student, Course  
4 ORDER BY Student.name, Course.title;  
5 """)
```

```
1 # 4 students . 4 courses = 16 rows
```

This query ignores Enrollment, so it has no idea who actually takes which course.

Correct version: use the relationship table

```
1 show("""
2 SELECT s.name, c.title
3 FROM Enrollment AS e
4 JOIN Student AS s ON e.student_id = s.id
5 JOIN Course AS c ON e.course_code = c.code
6 ORDER BY s.name, c.title;
7 """)
```

The relationship table removes the meaningless pairs and keeps only real enrollments.

Mental model for joins

1. Start with rows from the `FROM` table.
2. For each row, look for matching rows in the joined table.
3. Use the `ON` condition to decide what matches.
4. Build result rows from the matched data.
5. Apply `WHERE`, then sort with `ORDER BY`.

Your turn

Write a SQL query for each question.

1. Show each student name with each course code they take.
2. Show each student name with each course title they take.
3. Show all students enrolled in Calculus.
4. Show all enrollments in the CS department.

Use JOIN, ON, aliases, and the Enrollment table.

AGGREGATION

From rows to summaries

```
1 show("""  
2 SELECT student_id, course_code, grade  
3 FROM Enrollment;  
4 """)
```

Aggregation means computing one summary value from many rows.

Aggregate function: COUNT

```
1 SELECT COUNT(*)  
2 FROM Enrollment;
```

- ▶ COUNT(*) counts rows.
- ▶ The result is one number.
- ▶ Here, it counts all enrollments.

Count enrollments

```
1 show("""  
2 SELECT COUNT(*)  
3 FROM Enrollment;  
4 """)
```

Expected output:

```
1 (5,)
```

Aggregate function: AVG

```
1 SELECT AVG(grade)
2 FROM Enrollment;
```

- ▶ AVG computes the average of a numeric column.
- ▶ NULL values are ignored.
- ▶ The result is one summary value.

Average grade

```
1 show("""  
2 SELECT AVG(grade)  
3 FROM Enrollment;  
4 """)
```

Expected output:

```
1 (16.1,)
```

Aggregate functions: MIN and MAX

```
1 SELECT MIN(grade), MAX(grade)
2 FROM Enrollment;
```

- ▶ MIN returns the smallest value.
- ▶ MAX returns the largest value.
- ▶ They are often used with grades, prices, dates, and measurements.

Lowest and highest grade

```
1 show("""  
2 SELECT MIN(grade), MAX(grade)  
3 FROM Enrollment;  
4 """)
```

Expected output:

```
1 (14.0, 18.0)
```

Naming computed columns: AS

```
1 SELECT COUNT(*) AS number_of_enrollments  
2 FROM Enrollment;
```

- ▶ AS can name a computed result column.
- ▶ The name exists only in the query output.
- ▶ Aliases make results easier to read.

Readable aggregate result

```
1 show("""  
2 SELECT  
3     COUNT(*) AS number_of_enrollments,  
4     AVG(grade) AS average_grade  
5 FROM Enrollment;  
6 """)
```

Expected output:

```
1 (5, 16.1)
```

Aggregation after filtering

Question: what is the average grade in CS200?

```
1 show("""  
2 SELECT AVG(grade)  
3 FROM Enrollment  
4 WHERE course_code = 'CS200';  
5 """)
```

Expected output:

```
1 (17.5,)
```


SQL keyword: GROUP BY

```
1 SELECT course_code, COUNT(*)  
2 FROM Enrollment  
3 GROUP BY course_code;
```

- ▶ GROUP BY divides rows into groups.
- ▶ Each group gets one output row.
- ▶ Aggregate functions summarize each group separately.

Count enrollments per course

```
1 show("""
2 SELECT course_code, COUNT(*) AS number_of_students
3 FROM Enrollment
4 GROUP BY course_code;
5 """)
```

Expected output:

```
1 ('CS150', 1)
2 ('CS200', 2)
3 ('MA101', 2)
```

Average grade per course

```
1 show("""
2 SELECT course_code, AVG(grade) AS average_grade
3 FROM Enrollment
4 GROUP BY course_code;
5 """)
```

Expected output:

```
1 ('CS150', 15.5)
2 ('CS200', 17.5)
3 ('MA101', 15.0)
```

Average grade per course title

```
1 show("""
2 SELECT c.title, AVG(e.grade) AS average_grade
3 FROM Enrollment AS e
4 JOIN Course AS c ON e.course_code = c.code
5 GROUP BY c.code, c.title
6 ORDER BY average_grade DESC;
7 """)
```

Expected output:

```
1 ('Algorithms', 17.5)
2 ('Programming', 15.5)
3 ('Calculus', 15.0)
```

Where WHERE acts

```
1 SELECT course_code, AVG(grade)
2 FROM Enrollment
3 WHERE grade >= 16
4 GROUP BY course_code;
```

- ▶ WHERE filters individual rows.
- ▶ It happens before grouping.
- ▶ Here, low-grade rows are removed before averages are computed.

Filtering before grouping

```
1 show("""
2 SELECT course_code, AVG(grade) AS average_grade
3 FROM Enrollment
4 WHERE grade >= 16
5 GROUP BY course_code;
6 """)
```

Expected output:

```
1 ('CS200', 17.5)
2 ('MA101', 16.0)
```

WHERE vs grouped results

Clause	Role
WHERE	filters rows before groups are formed
GROUP BY	forms groups from the remaining rows
Aggregate functions	compute one value per group

Use WHERE when the condition concerns individual rows.

Why this query is wrong

```
1 SELECT course_code, COUNT(*) AS n
2 FROM Enrollment
3 WHERE COUNT(*) >= 2
4 GROUP BY course_code;
```

WHERE cannot filter using COUNT(*) because groups do not exist yet.

Full aggregation pattern

```
1 SELECT group_column, aggregate_function(column)
2 FROM table_name
3 WHERE row_condition
4 GROUP BY group_column
5 ORDER BY aggregate_result;
```

- ▶ WHERE is optional.
- ▶ HAVING is optional.
- ▶ GROUP BY is needed when summaries are computed per group.

Your turn

Write a SQL query for each question.

1. Count all enrollments.
2. Compute the average grade in Enrollment.
3. For each course code, show the number of enrolled students.
4. For each course title, show the average grade.

Use COUNT, AVG, and GROUP BY.

INDEXING BASICS

Why indexing matters

- ▶ A database table may contain millions of rows.
- ▶ Some queries should not scan every row.
- ▶ An index helps the DBMS find matching rows faster.

An index is an extra data structure built to speed up lookup.

Table scan vs index lookup

Method	Idea
Table scan	inspect rows one by one until the query is answered
Index lookup	use an auxiliary structure to jump closer to matching rows

A table scan is like searching a list. An index lookup is closer to searching through a dictionary or tree.

A query that may scan a table

```
1 show("""  
2 SELECT id, name, major  
3 FROM Student  
4 WHERE major = 'CS';  
5 """)
```

If there is no useful index on `major`, SQLite may inspect the table rows to find matching students.

SQL command: EXPLAIN QUERY PLAN

```
1 EXPLAIN QUERY PLAN
2 SELECT id, name, major
3 FROM Student
4 WHERE major = 'CS';
```

- ▶ EXPLAIN QUERY PLAN asks SQLite how it plans to run a query.
- ▶ It does not return the query result.
- ▶ It returns information about the execution strategy.

Inspect a query plan from Python

```
1 show("""  
2 EXPLAIN QUERY PLAN  
3 SELECT id, name, major  
4 FROM Student  
5 WHERE major = 'CS';  
6 """)
```

Possible output before indexing:

```
1 (2, 0, 0, 'SCAN Student')
```


Reading SCAN Student

```
1 (2, 0, 0, 'SCAN Student')
```

- ▶ SCAN Student means SQLite scans the Student table.
- ▶ For a tiny table, this is fine.
- ▶ For a large table, repeated scans can become expensive.

SQL command: CREATE INDEX

```
1 CREATE INDEX idx_student_major  
2 ON Student(major);
```

- ▶ CREATE INDEX builds an index.
- ▶ idx_student_major is the index name.
- ▶ Student(major) means the index is built on the major column.

Create an index from Python

```
1 con.execute("""  
2 CREATE INDEX idx_student_major  
3 ON Student(major);  
4 """)
```

The table data does not change. SQLite only creates an additional lookup structure.

Inspect the query plan again

```
1 show("""  
2 EXPLAIN QUERY PLAN  
3 SELECT id, name, major  
4 FROM Student  
5 WHERE major = 'CS';  
6 """)
```

Possible output after indexing:

```
1 (3, 0, 0, 'SEARCH Student USING INDEX idx_student_major (major=?)')
```

Reading SEARCH ... USING INDEX

```
1 SEARCH Student USING INDEX idx_student_major (major=?)
```

- ▶ SEARCH means SQLite is using a more targeted lookup.
- ▶ USING INDEX idx_student_major names the chosen index.
- ▶ major=? means the index helps with filtering by major.

Where indexes help

Indexes often help columns used in:

- ▶ WHERE filters;
- ▶ JOIN matching conditions;
- ▶ ORDER BY sorting, in some cases;
- ▶ frequent lookup queries.

A useful index matches the way the application actually queries the data.

Index for enrollment lookup

Question: find all courses taken by student 3.

```
1 show("""  
2 SELECT course_code, grade  
3 FROM Enrollment  
4 WHERE student_id = 3;  
5 """)
```

Candidate index:

```
1 CREATE INDEX idx_enrollment_student  
2 ON Enrollment(student_id);
```

Create the enrollment index

```
1 con.execute("""
2 CREATE INDEX idx_enrollment_student
3 ON Enrollment(student_id);
4 """)
```

This index supports queries that look up enrollments by student.

Index for course lookup

Question: find all students enrolled in CS200.

```
1 show("""
2 SELECT student_id, grade
3 FROM Enrollment
4 WHERE course_code = 'CS200';
5 """)
```

Candidate index:

```
1 CREATE INDEX idx_enrollment_course
2 ON Enrollment(course_code);
```

Indexes and joins

```
1 SELECT s.name, c.title, e.grade
2 FROM Enrollment AS e
3 JOIN Student AS s ON e.student_id = s.id
4 JOIN Course AS c ON e.course_code = c.code;
```

- ▶ Joins match foreign keys to primary keys.
- ▶ Primary keys are indexed automatically in SQLite.
- ▶ Extra indexes on foreign-key columns can help join performance.

Index tradeoffs

Benefit	faster reads for matching queries
Cost	extra storage space
Cost	slower inserts, updates, and deletes
Risk	useless indexes add cost without helping queries

Indexes are not free. Do not index every column blindly.

When to consider an index

Consider creating an index when:

- ▶ the table is large;
- ▶ the query is frequent;
- ▶ the column appears often in WHERE;
- ▶ the column appears often in JOIN ... ON;
- ▶ the performance problem is measured, not guessed.

When an index may be useless

An index may not help much when:

- ▶ the table is tiny;
- ▶ the query returns most rows anyway;
- ▶ the indexed column is rarely used in filters or joins;
- ▶ the workload mostly inserts and updates data;
- ▶ there are already suitable primary-key indexes.

Bad habit: indexing everything

```
1 CREATE INDEX idx_student_name ON Student(name);  
2 CREATE INDEX idx_student_major ON Student(major);  
3 CREATE INDEX idx_course_title ON Course(title);  
4 CREATE INDEX idx_course_credits ON Course(credits);  
5 CREATE INDEX idx_course_department ON Course(department);
```

This looks serious, but it may be wasteful. Indexes should follow real query needs.

Better habit: index for a query

Query:

```
1 SELECT name
2 FROM Student
3 WHERE major = 'CS';
```

Useful index:

```
1 CREATE INDEX idx_student_major
2 ON Student(major);
```

Start from the query. Then decide whether an index supports it.

Basic indexing pattern

```
1 EXPLAIN QUERY PLAN
2 SELECT ...
3 FROM ...
4 WHERE ...;
5
6 CREATE INDEX index_name
7 ON table_name(column_name);
8
9 EXPLAIN QUERY PLAN
10 SELECT ...
11 FROM ...
12 WHERE ...;
```

Inspect before and after. Do not assume the index is being used.

Your turn

For each query, suggest whether an index may help.

1. `SELECT * FROM Student WHERE major = 'CS';`
2. `SELECT * FROM Enrollment WHERE student_id = 3;`
3. `SELECT * FROM Course;`
4. `SELECT * FROM Course WHERE department = 'CS';`

Justify the index using the query, not by saying “indexes are faster.”

RECAP

The SQL query structure

A SQL query is built from clauses. Each clause has a role.

Clause	Role
SELECT	choose output columns
FROM	choose source table
JOIN ... ON	combine related tables
WHERE	filter rows
GROUP BY	form groups
ORDER BY	sort results
LIMIT	keep only a fixed number of rows

Full query pattern

```
1 SELECT selected_columns
2 FROM table_name
3 JOIN other_table ON matching_condition
4 WHERE row_condition
5 GROUP BY grouping_columns
6 ORDER BY sorting_columns
7 LIMIT n;
```

Not every query needs every clause. Start with the question, then choose the clauses needed to answer it.

Joins recombine normalized tables

- ▶ Normalization separates facts into clean tables.
- ▶ `Student` stores student facts.
- ▶ `Course` stores course facts.
- ▶ `Enrollment` stores the relationship.
- ▶ `JOIN` recombines them when a query needs data from several tables.

We split data for correctness. We join data to answer questions.

Join recap

```
1 SELECT s.name, c.title, e.grade
2 FROM Enrollment AS e
3 JOIN Student AS s ON e.student_id = s.id
4 JOIN Course AS c ON e.course_code = c.code;
```

- ▶ Enrollment gives the pairs.
- ▶ Student gives names.
- ▶ Course gives titles.
- ▶ ON gives the matching rule.

Grouping summarizes rows

- ▶ Aggregation turns many rows into summary values.
- ▶ COUNT counts rows.
- ▶ AVG computes an average.
- ▶ MIN and MAX find extremes.
- ▶ GROUP BY computes summaries per group.

Without GROUP BY: one summary for the whole table. With GROUP BY: one summary per group.

Grouping recap

```
1 SELECT c.title, COUNT(*) AS number_of_students
2 FROM Enrollment AS e
3 JOIN Course AS c ON e.course_code = c.code
4 GROUP BY c.code, c.title;
```

This query gives one result row per course.

Ordering is explicit

- ▶ Tables do not have inherent order.
- ▶ Query results should not be trusted to appear in a stable order.
- ▶ Use `ORDER BY` when order matters.
- ▶ Use `LIMIT` when only the first few sorted rows are needed.

If the query does not say `ORDER BY`, the result is not ordered.

Ordering recap

```
1 SELECT title, credits
2 FROM Course
3 ORDER BY credits DESC, title ASC
4 LIMIT 2;
```

- ▶ Sort first by credits, highest first.
- ▶ Break ties using title.
- ▶ Keep only two rows.

Indexes speed reads at a cost

- ▶ An index is an extra lookup structure.
- ▶ It can speed up queries using `WHERE`, `JOIN`, or sometimes `ORDER BY`.
- ▶ It costs storage space.
- ▶ It can slow down inserts, updates, and deletes.

Indexes are useful when they support real frequent queries. They are not something to add blindly.

Indexing recap

```
1 EXPLAIN QUERY PLAN
2 SELECT id, name, major
3 FROM Student
4 WHERE major = 'CS';
5
6 CREATE INDEX idx_student_major
7 ON Student(major);
```

Use EXPLAIN QUERY PLAN to inspect whether SQLite scans the table or uses an index.

What you should be able to do now

- ▶ Query rows using `SELECT`, `FROM`, and `WHERE`.
- ▶ Handle missing values using `IS NULL`.
- ▶ Sort and limit results using `ORDER BY` and `LIMIT`.
- ▶ Join normalized tables using `JOIN ... ON`.
- ▶ Summarize rows using aggregate functions and `GROUP BY`.
- ▶ Reason about basic indexes and their tradeoffs.

Exit ticket

Answer briefly.

1. Why do normalized schemas often require joins?
2. Why should we not trust row order without `ORDER BY`?
3. What is one benefit and one cost of an index?

Use the university schema in your answers: `Student`, `Course`, and `Enrollment`.